COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	25/08/2026	**DATE SUBMITTED:**	08/25/2026
NAME:	Alvarez, Erick Jefferson A	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Ansible Basics

1. PROCEDURE OUTPUT

Task 1: Run elevated ad hoc commands

1. Locally, we use the command *sudo apt update* when we want to download package information from all configured resources. The sources are often defined in
   /etc/apt/sources.list file and other files located in
   /etc/apt/sources.list.d/ directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

```
erix@erickalvarez:~$ ansible all -i inventory.ini -m  apt -a update_cache=true
192.168.56.103 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "msg": "Failed to lock apt for exclusive operation: Failed to lock directo
 /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - ope
(13: Permission denied)"
}
192.168.56.104 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "msg": "Failed to lock apt for exclusive operation: Failed to lock directo
 /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - ope
(13: Permission denied)"
}
```

What is the result of the command? Is it successful?

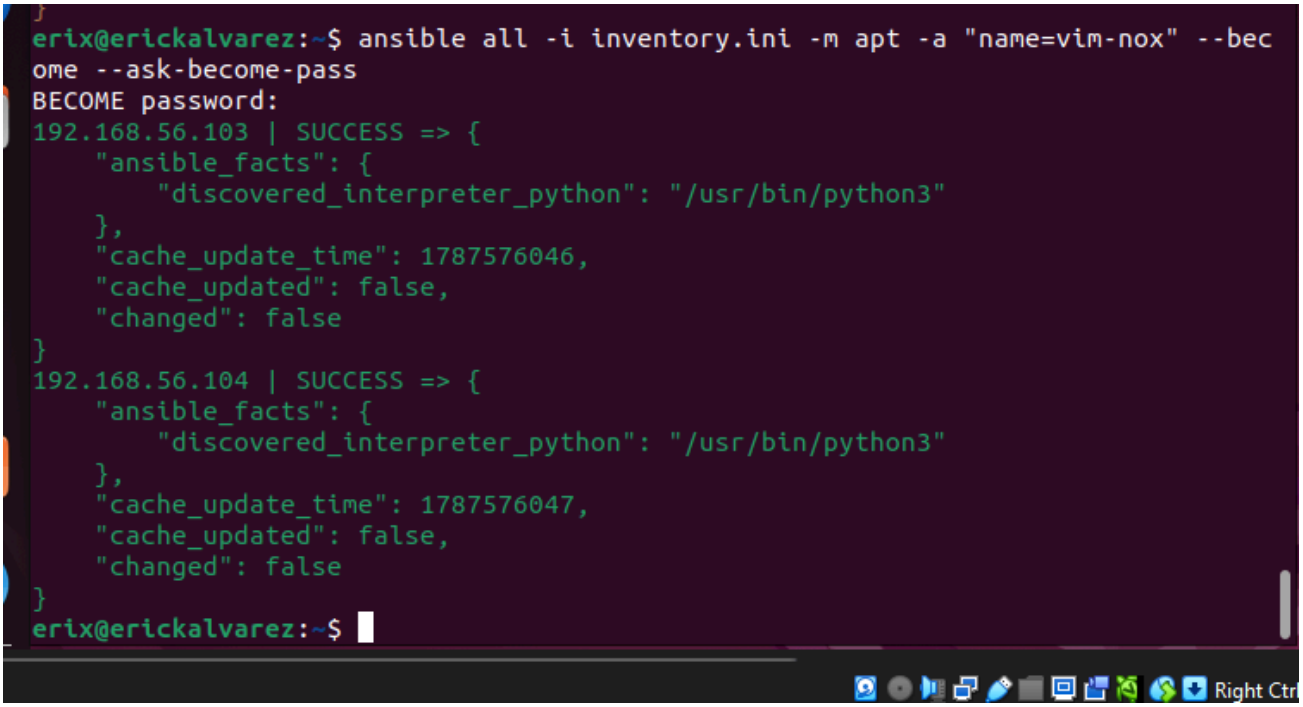
- The command failed because the apt needs administrator permission

Try editing the command and add something that would elevate the privilege. Issue the command *ansible all -m apt -a update_cache=true --become --ask-become-pass*. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The *update_cache=true* is the same thing as running sudo apt update. The *--become* command elevates the privileges and the *--ask-become-pass* asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is

CHANGED compared to the first command, which is **FAILED**.

```
erix@erickalvarez:~$ ansible all -i inventory.ini -m apt -a update_cache=true --
become --ask-become-pass
BECOME password:
192.168.56.103 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787576046,
    "cache_updated": true,
    "changed": true
}
192.168.56.104 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787576047,
    "cache_updated": true,
    "changed": true
}
erix@erickalvarez:~$
```

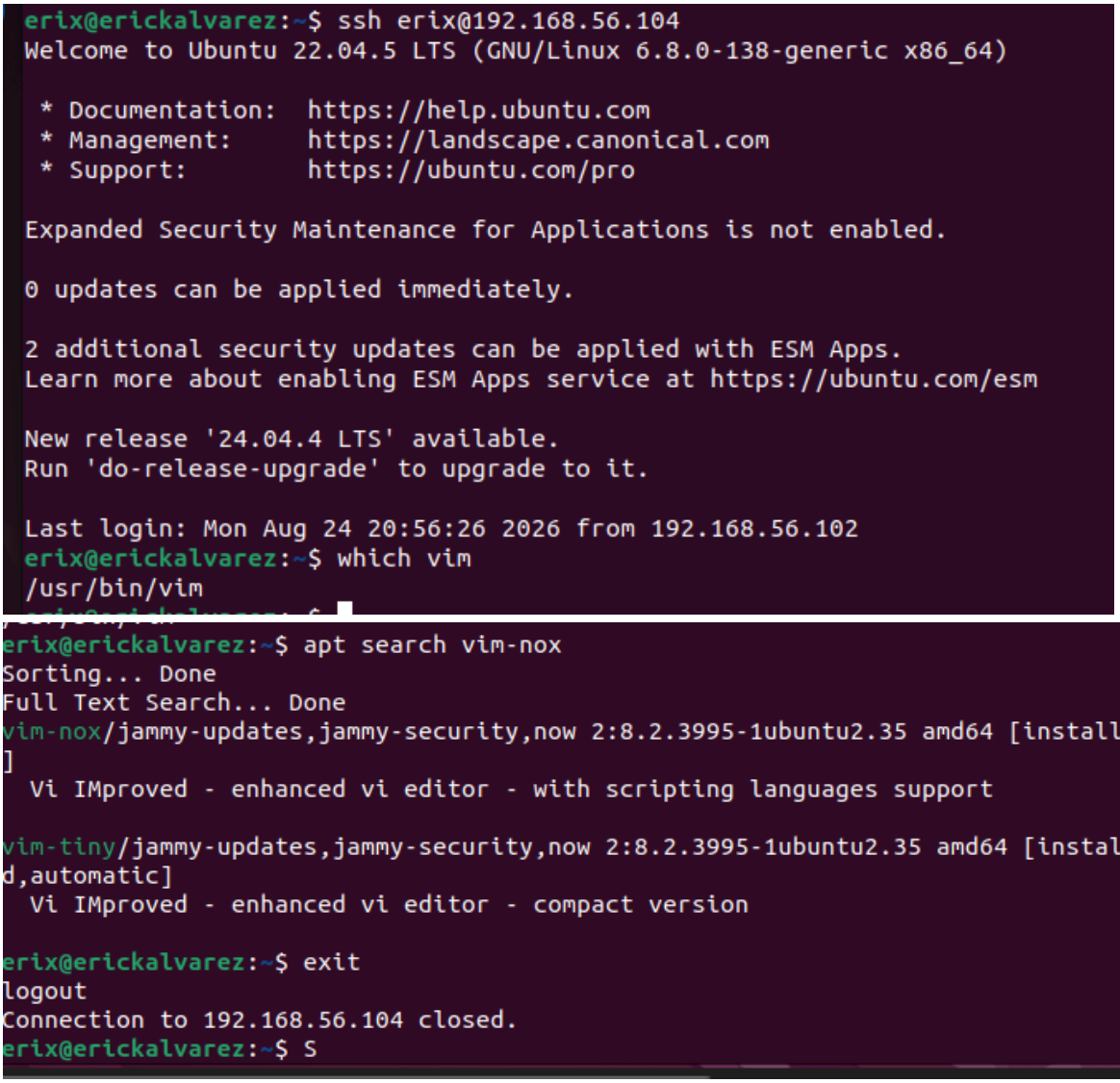
2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: *ansible all -m apt -a name=vim-nox --become --ask-become-pass*. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.



a. Verify that you have installed the package in the remote servers. Issue the command

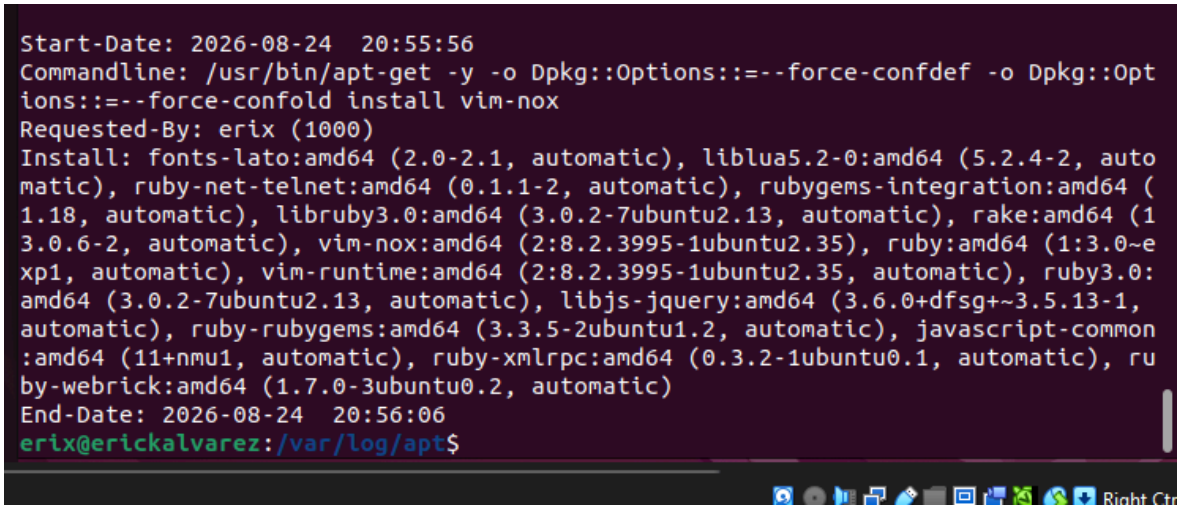
which vim

and the command *apt search vim-nox* respectively.



Was the command successful?

- Yes, my output shows that the installation of vim on both nodes are successful



b. Check the logs in the servers using the following commands: *cd /var/log*. After this, issue the command *ls*, go to the folder *apt* and open *history.log*.

Describe what you see in the history.log.

- It shows the package activities done on the server, such as installing packages.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

a. Issue the command: *ansible all -m apt -a name=snapd --become --ask-become-pass*



Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?.

- The command successfully checks and installs snapd on both remote servers. If the snapd is already installed on both remote servers, ansible may report no changes

b. Now, try to issue this command: *ansible all -m apt -a "name=snapd state=latest"* *--become* *--ask-become-pass*

```
erix@erickalvarez:~$ ansible all -i inventory.ini -m apt -a "name=snapd state=la
test" --become --ask-become-pass
BECOME password:
192.168.56.104 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787576047,
    "cache_updated": false,
    "changed": false
}
192.168.56.103 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "cache_update_time": 1787576046,
    "cache_updated": false,
    "changed": false
}
erix@erickalvarez:~$
```

Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

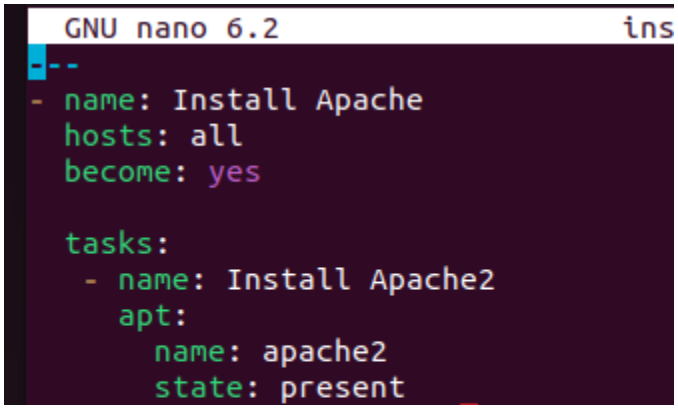
- It checks if snapd is updated. If it is not the latest version, Ansible updates it.

Task 2: Writing our First Playbook

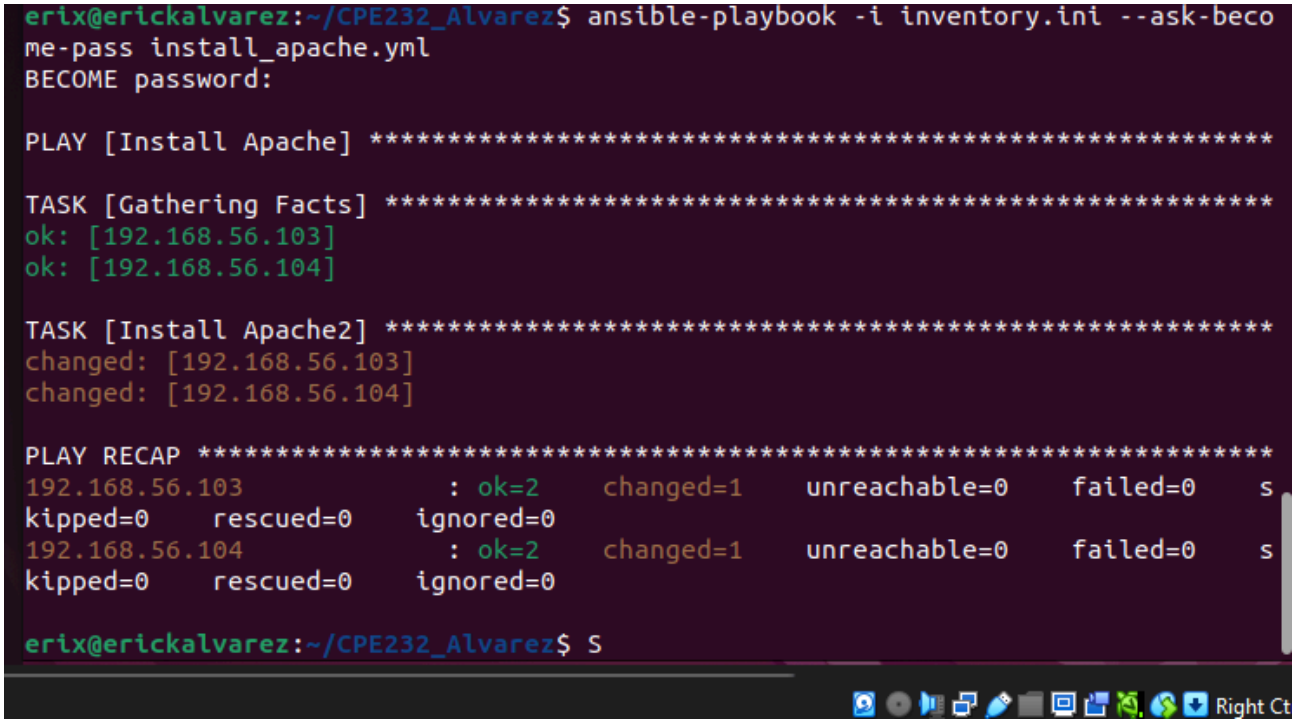
1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command *nano install_apache.yml*. This will

create a playbook file called *install_apache.yaml*. The .yaml is the basic standard extension for playbook files.

When the editor appears, type the following:



2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml.*

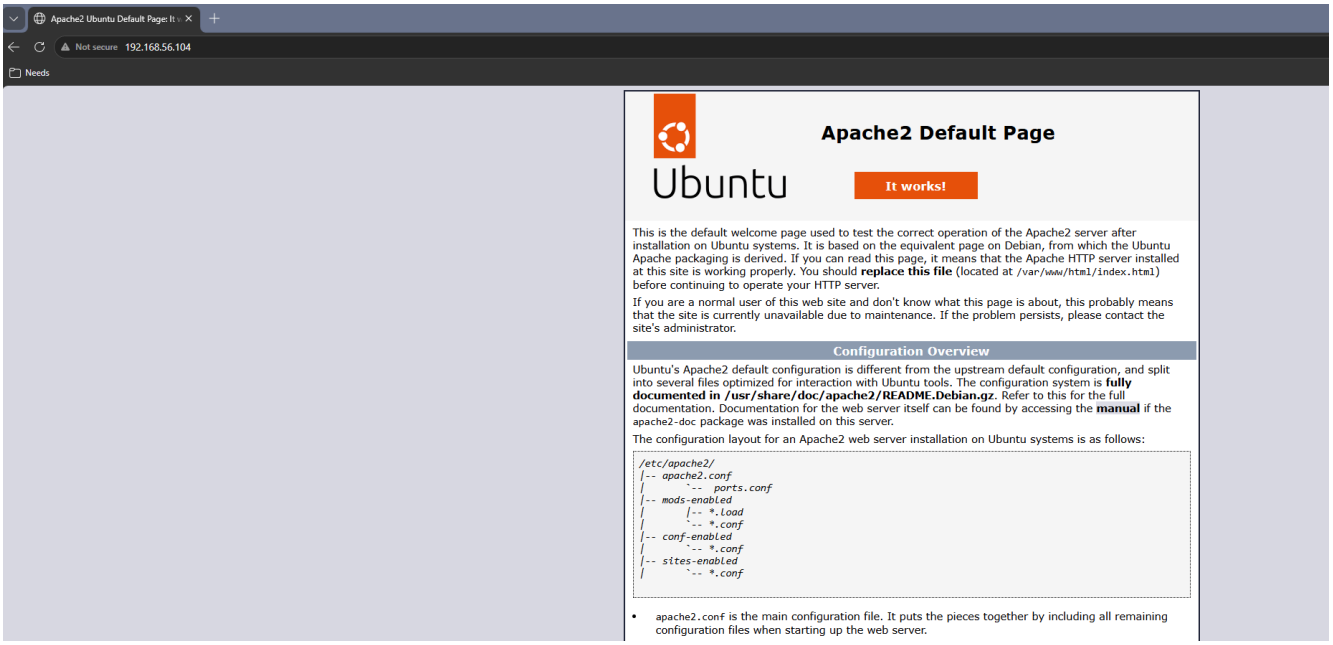


Describe the result of this command.

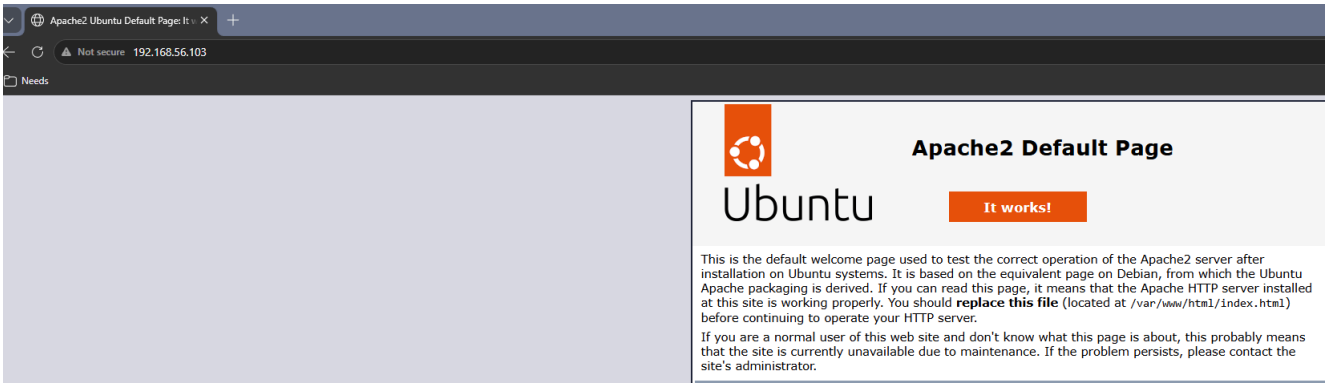
- The playbook successfully connects to the managed nodes and installs Apache2. The PLAY RECAP should show the number of successful tasks and should have failed=0. If Apache was already installed, the task may show ok instead of changed.

3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.

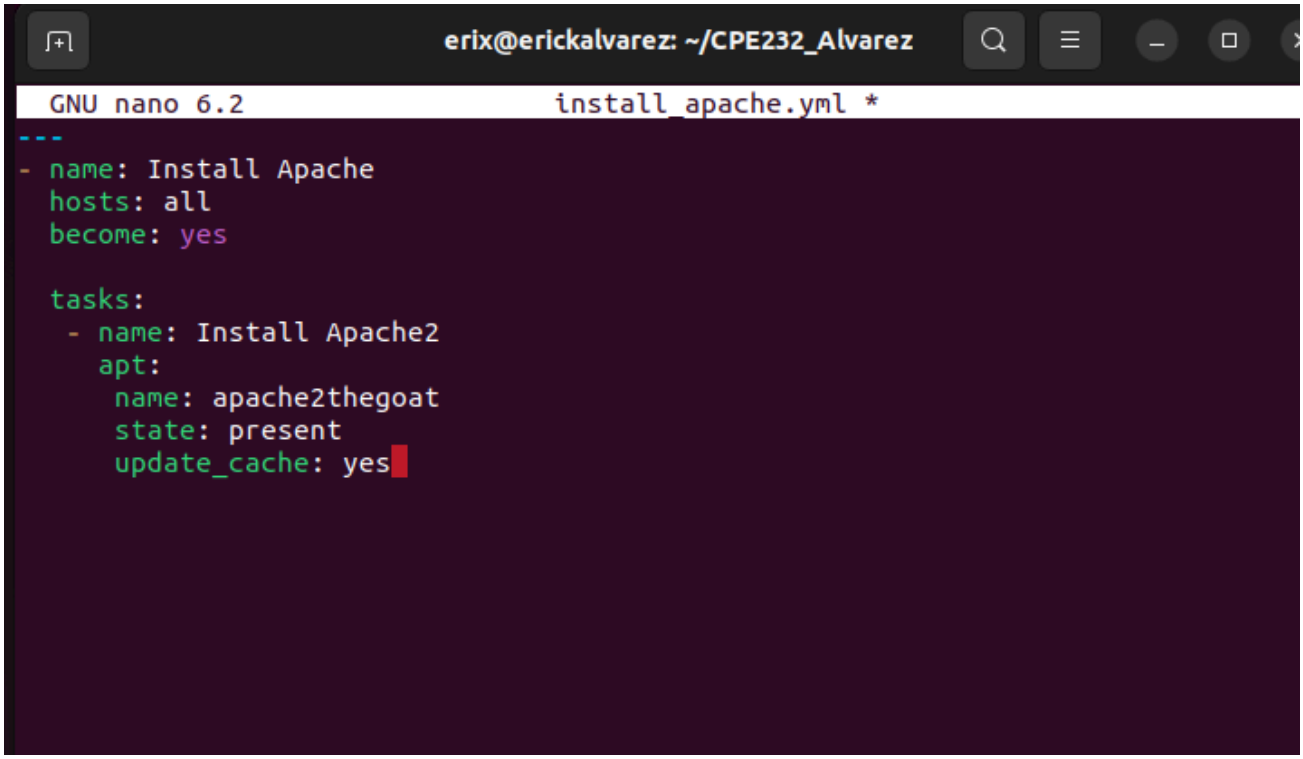
192.168.56.104



192.168.56.103



4.
Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized.


erix@erickalvarez: ~/CPE232_Alvarez
GNU nano 6.2 install_apache.yml *

- name: Install Apache
hosts: all
become: yes
tasks:
- name: Install Apache2
apt:
name: apache2thegoat
state: present
update_cache: yes

```
erix@erickalvarez:~/CPE232_Alvarez$ ansible-playbook -i inventory.ini --ask-beco
me-pass install_apache.yml
BECOME password:

PLAY [Install Apache] ***************************************************

TASK [Gathering Facts] **************************************************
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [Install Apache2] **************************************************
fatal: [192.168.56.103]: FAILED! => {"changed": false, "msg": "No package matchi
ng 'apache2thegoat' is available"}
fatal: [192.168.56.104]: FAILED! => {"changed": false, "msg": "No package matchi
ng 'apache2thegoat' is available"}

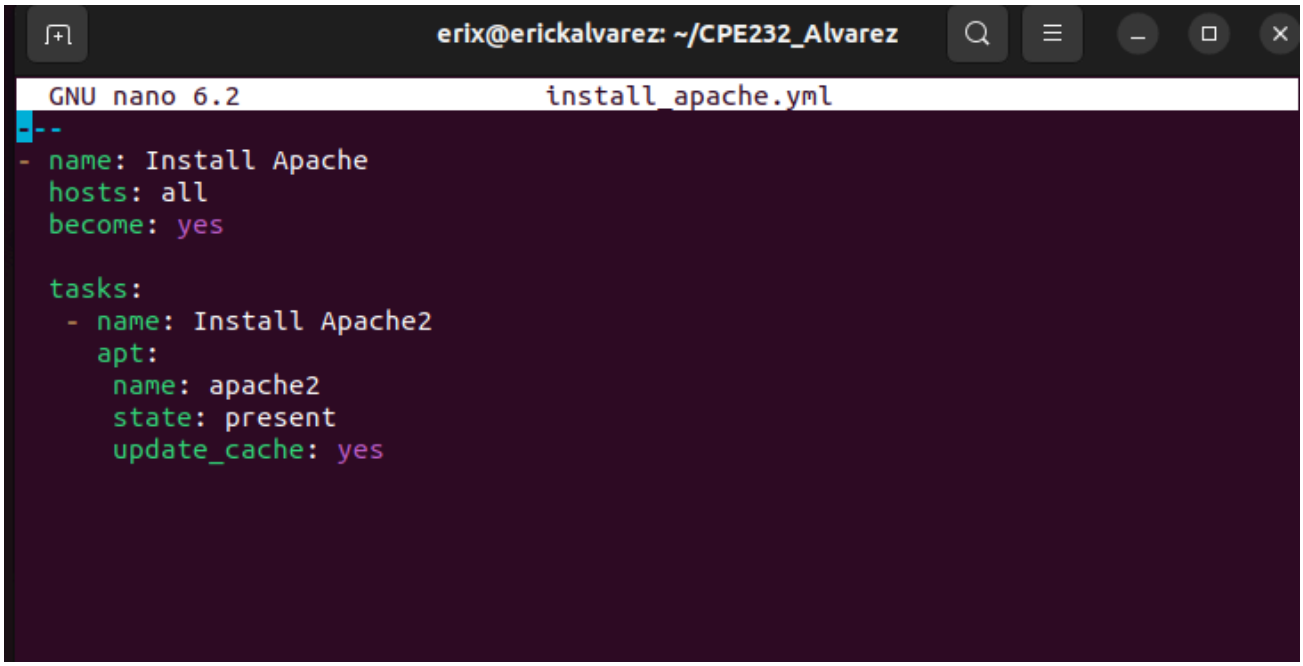
PLAY RECAP **************************************************************
192.168.56.103             : ok=1    changed=0    unreachable=0    failed=1    s
kipped=0    rescued=0    ignored=0
192.168.56.104             : ok=1    changed=0    unreachable=0    failed=1    s
kipped=0    rescued=0    ignored=0

erix@erickalvarez:~/CPE232_Alvarez$
```

What is the output?

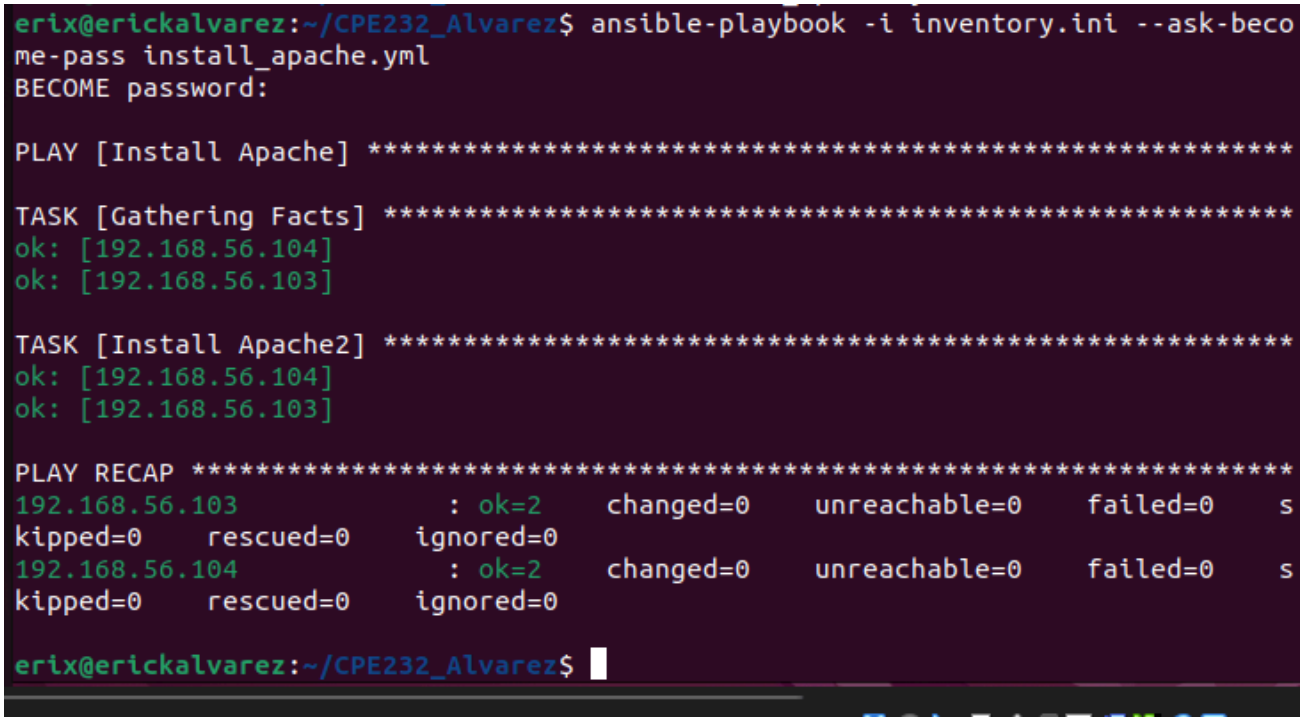
- Since I changed the name of the task. Installing apache and gathering hosts are successful but when installing apache2, it showed that no package matched ‘apache2thegoat’

5. This time, we are going to put additional tasks into our playbook. Edit the *install_apache.yml*. As you can see, we are now adding an additional command, which is the *update_cache*. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.



Save the changes to this file and exit.

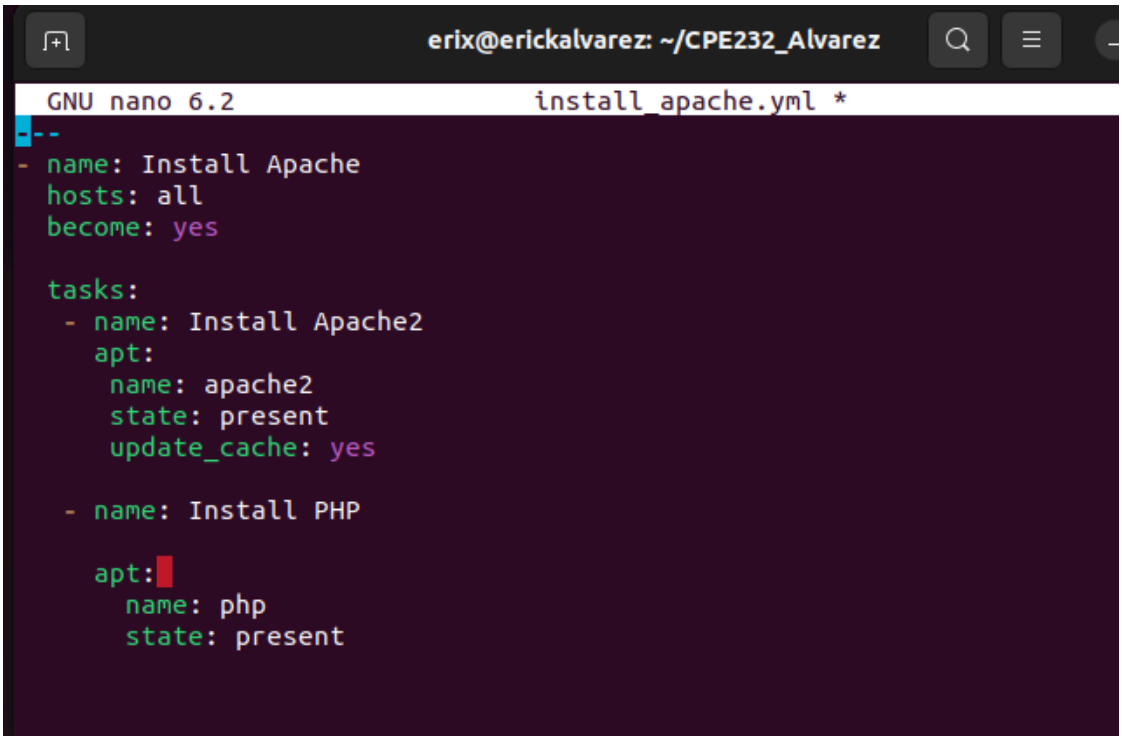
6. Run the playbook and describe the output.

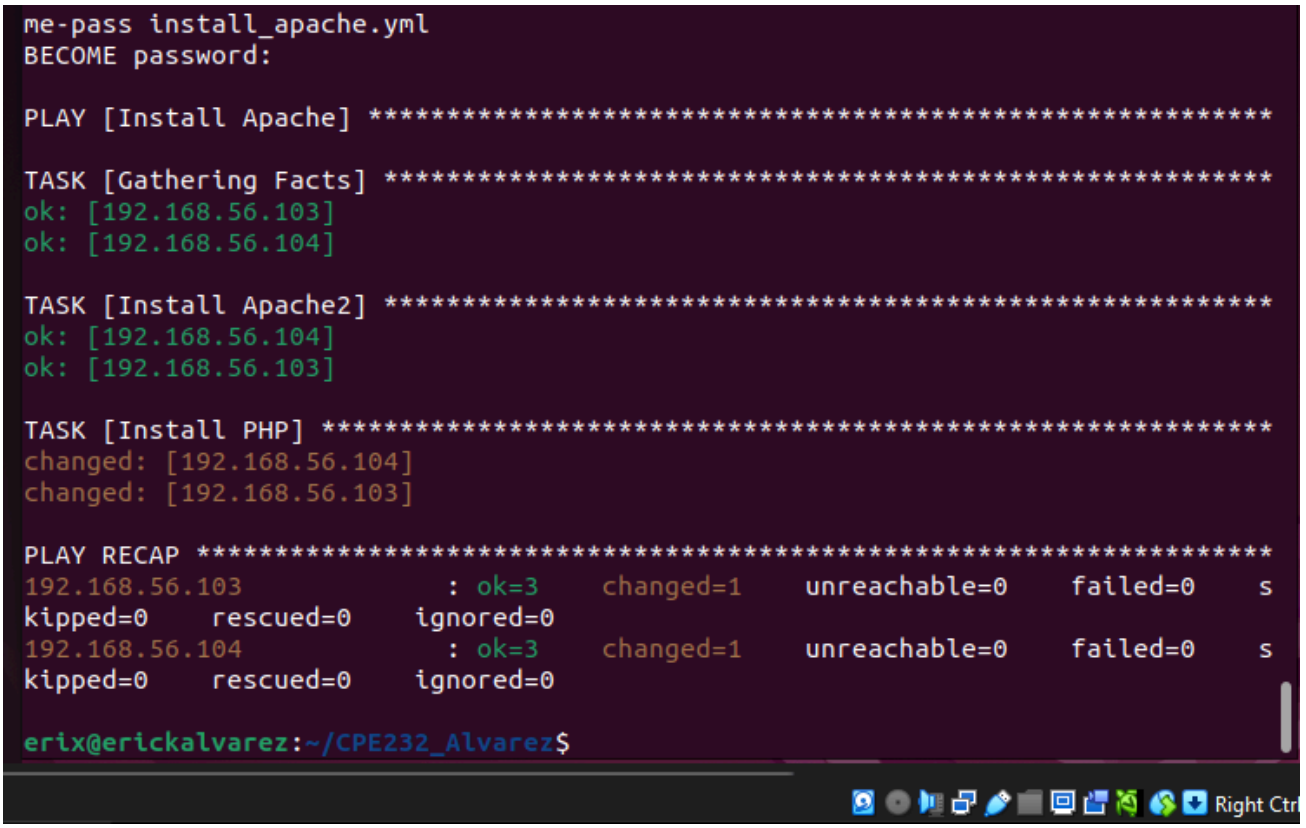


Did the new command change anything on the remote servers?

- It updates the package list before installing Apache. It may show CHANGED if an update was needed.

7. Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier.



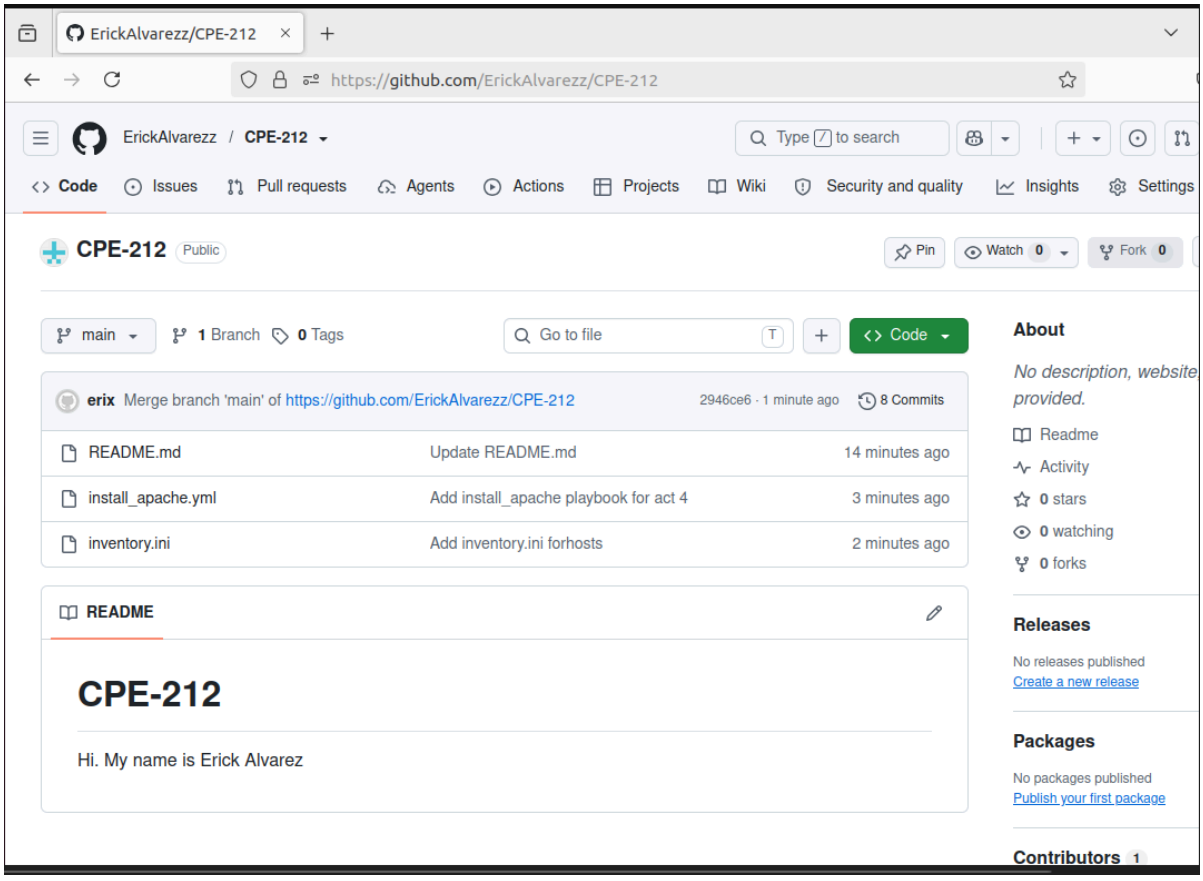


Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?
   - Yes, if PHP was not installed, Ansible installs it on the remote servers.

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

```
 1 file changed, 2 insertions(+), 2 deletions(-)
erix@erickalvarez:~/CPE232_Alvarez/CPE-212$ git push origin main
Username for 'https://github.com': ErickAlvarezz
Password for 'https://ErickAlvarezz@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.05 KiB | 1.05 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/ErickAlvarezz/CPE-212
   c4b3576..2946ce6  main -> main
erix@erickalvarez:~/CPE232_Alvarez/CPE-212$
```



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?
   - The importance of playbooks is that you have the opportunity to create a file with the desired state of your servers once and use it for as many servers as needed. You don't have to enter the command multiple times; you have to execute the playbook and then all those actions will be executed one after another. The main thing is that the process is reproducible as Ansible changes only necessary things and it is just a text file which can be committed into Git.

2. Summarize what we have done on this activity.
   - At the very beginning, we ran ad hoc commands with privilege escalation using --become --ask-become-pass and performed such actions as updating package cache and installing vim-nox on the remote servers. We also tried installing a package by name and using state=latest, which means installing the latest version of the package. Then we created our first playbook called install_apache.yml. At first, we added installing of Apache2, then we added the step of updating the cache, and then – PHP. We checked every new version of the playbook by running it and looking at the Apache default page in the browser. The activity allowed understanding that the playbooks are a kind of a more convenient version of ad hoc commands.

Conclusions/Learnings:

This activity required far more debugging than I thought, particularly on connecting and working with Ansible of both the managed nodes. There were a number of problems that I encountered in syncing the work with GitHub, including the problems of being in the wrong directory, pathspec errors as the file wasn't located where I thought, and branching divergence while attempting to push. While it took me a while to debug each of these error messages, I gained quite a few insights on how Git really works as opposed to just how to execute the commands. In all, even with the difficulties, I understand Ansible privilege escalation and playbooks, as well as Git.

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used only for explanation of concepts, especially for handling errors like git and connecting nodes.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.